

Multi-Piano: LAN Multiplayer Terminal Piano

CMPE487

Ömer Faruk Erzurumluoğlu Muhammet Berkay Keskin

Department of Computer Engineering

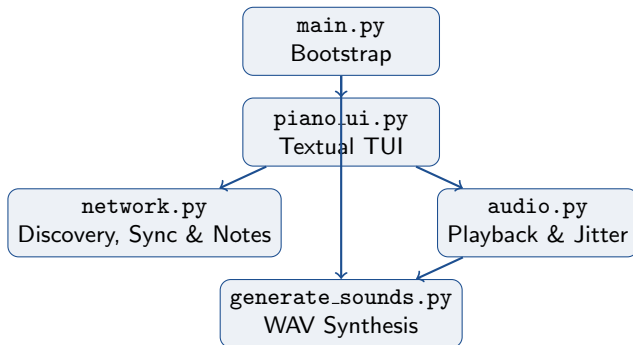
June 2026

- 1 Project Overview
- 2 System Architecture
- 3 Networking
- 4 Audio Pipeline
- 5 User Interface
- 6 Deployment & Future Work

Multi-Piano is a LAN-based collaborative piano application with a terminal user interface (TUI).

- Multiple players on the same network discover, host, and join lobbies.
- Notes are transmitted in near real time and played back with **clock-synchronised** audio.
- No external sound assets: piano tones are synthesized procedurally at build or first run.
- A 13-key one-octave keyboard (C4–C5) is rendered as an ASCII piano in the terminal.

Goal: demonstrate a lightweight multiplayer prototype combining discovery, mixed protocols, NTP-style clock sync, scheduled payout, and a full TUI.



- MultiPianoApp owns AudioEngine, NetworkManager, and JitterBuffer.
- Network I/O runs on daemon threads; UI updates use `call_from_thread()`.

We separate concerns into three network channels:

Port	Protocol	Format	Role
12488	UDP	JSON	Lobby discovery (broadcast)
12487	TCP	length-prefixed JSON	Lobby management
12489	UDP	binary + JSON	Note events & clock sync

- **Discovery** finds open sessions without prior configuration.
- **TCP** maintains player membership and state synchronisation.
- **UDP 12489** carries 15-byte note packets and NTP-style SYNC_REQ/SYNC_ACK messages.

Client behaviour:

- Binds UDP port 12488 with `SO_BROADCAST` and `SO_REUSEADDR`.
- Sends `DISCOVER` broadcasts; listens for `LOBBY_REPLY` responses.

Host behaviour:

- Replies to discoverers with lobby name, host IP, TCP port, and active player count.
- Re-broadcasts `LOBBY_REPLY` every 2 seconds.

Stale lobby eviction: entries expire after 6 seconds without a reply (cleanup every 3 s).

All TCP messages use a **4-byte big-endian length prefix** followed by UTF-8 JSON.

Join flow:

- 1 Client connects to (`host_ip`, 12487).
- 2 Client sends `{"type": "JOIN", "SENDER_NAME": ..., "SENDER_IP": ...}`.
- 3 Host accepts, stores the socket, and appends the client IP to `peer_ips`.
- 4 Host broadcasts `STATE_UPDATE` with the current player list to all clients.
- 5 Both sides start a clock-sync loop (every 5 s) over UDP 12489.

Disconnect is detected via TCP EOF; the lobby list is updated and the UI is notified.

Note packets (15 bytes, little-endian): seq | play_at (float64) | note_id | action | velocity

Sender scheduling:

- On key press, stamp `play_at = time.time() + 30ms`.
- Local and remote playback both target this absolute time.

NTP-style clock sync (JSON on UDP 12489):

- SYNC_REQ carries t_1 ; SYNC_ACK returns t_1, t_2, t_3 .
- Offset: $\text{offset} = \frac{(t_2 - t_1) + (t_3 - t_4)}{2}$ (peer clock – local clock).
- Receiver converts: `local_play_at = play_at - offset`.

Local playback:

- Key press schedules `play_note` and broadcast after **30 ms** (`PLAYOUT_DELAY`).
- Release uses the same delay so local and remote notes stay aligned.

JitterBuffer for remote notes:

- Priority queue keyed by a **monotonic deadline** (immune to wall-clock steps).
- Waits until the sender-scheduled play time, then plays or stops the note.
- Deduplicates by `(sender_ip, seq)`; drops events more than 200 ms overdue.
- Before first sync, offset is 0 — notes play ~ 30 ms after arrival (graceful fallback).

AudioEngine: pygame mixer, 32 channels, 200 ms fadeout, 3 s stuck-note

LobbyScreen: player name, lobby name, host/join/refresh, discovered-lobby list.

PianoScreen: ASCII piano with box-drawing borders, player sidebar, activity log.

Keyboard mapping: a s d f g h j k (white keys), w e t y u (black keys).

Input model: Textual has no key-up events — 500 ms timer auto-releases each note; key-repeat is debounced. Remote presses flash for 400 ms.

docker-compose provides a two-player LAN test environment:

- Base image: `python:3.12-slim` with SDL2 runtime libraries.
- Custom bridge network `172.28.0.0/24` with fixed container IPs.
- Sounds pre-generated at image build time for instant startup.
- Requires `stdin_open: true` and `tty: true` — TUI apps cannot run detached.

```
docker compose build
```

```
docker compose run --rm player1 (terminal 1)
```

```
docker compose run --rm player2 (terminal 2)
```

Current limitations:

- Hub topology: guests hear the host, but **not each other**.
- LAN-only discovery; no Internet/WAN support or NAT traversal.
- Fixed 500 ms auto-release approximates note-off rather than true key-up.
- Single octave (13 keys); no sustain pedal or velocity from key pressure.

Possible extensions:

- Full-mesh or host-relay note forwarding for guest-to-guest audio.
- Wider keyboard range, MIDI input, and configurable instrument sounds.

Thank you for your attention

Ömer Faruk Erzurumluoğlu
Muhammet Berkay Keskin

omer.erzurumluoglu@std.bogazici.edu.tr
muhammet.keskin@std.bogazici.edu.tr